AI 创新应用挑战赛 · 参赛材料

心屿（Isle）

开发与 AI 协同全记录（合订本）

一座会记住你的心灵小岛

面向大学生的 AI 情绪支持应用



沧州交通学院 · 计算机与信息技术学院 · 软件工程 2410 · 吴思彤

v2.3.4 · 2026-11-13

目录

阅读说明

合订说明

第一部分 · 立项与方案

1. 立项

2. 技术选型与关键决策

3. 早期开发期问题

4. 首个可用版本

第二部分 · 情绪支持重构

5. 评分复盘

6. 方向调整

7. 关键实现

8. 重构期问题

第三部分 · 体验与交付

9. 四栏导航

10. 启动封面

11. 新图标

12. 记忆折叠

13. 真机验证

第四部分 · AI 协同方法

14. Prompt 链示例

15. 排错闭环

16. 用户偏好

17. 经验总结

第五部分 · 密钥方案演进

18. 从“内置 Key”到 Serverless 代理，再回到本地配置

第六部分 · AI 对话与 Prompt 全记录

文档说明

阅读说明

全程时间线

序幕 · 立项：从方案探索到原生 Android（10–15 15:51—16:03）

第一幕 · 工程骨架与核心闭环（10–15 16:03—16:30）

第二幕 · 角色、资料与隐私（10–15 16:31—16:55）

第三幕 · 体验缺陷攻坚（10–15 16:55—17:18）

第四幕 · 液态玻璃与视觉重构（10–15 17:18—17:42）

第五幕 · 从代码到交付材料（10–16）

第六幕 · 未成年保护与最终交付（10–17）

第七幕 · 交付后修复（10–17 15:00—15:30）

成品状态

后续迭代 · 交付前收口（10–26～10–29）

后续迭代 · 从 v1.0.0 到 v2.3.4（2026–10–29 — 2026–11–13）

附录 · 关键文件

结语

阅读说明

这份文档很长，因为它保留了开发过程中的对话、报错和修改轨迹。

第一次读，建议先看「合订说明」和每部分开头的小结，了解项目是怎么一步步走到现在的； 如果你关心某个具体问题，再按目录跳到对应章节。第六部分是 AI 对话与 Prompt 全记录，属于证据材料，不需要从头读到尾。

合订说明

本记录分为六部分：

1. **立项与方案**：立项、技术选型、核心闭环、文档与交付；
2. **情绪支持重构**：角色重设、场景、多智能体安全、本地账户、记忆管理；
3. **体验与交付**：四栏导航、启动封面、新图标、记忆折叠、真机验证；
4. **AI 协同方法**：Prompt 链、排错闭环、用户偏好与经验总结；
5. **密钥方案演进**：从内置 Key 到代理尝试，再回到本地配置；
6. **AI 对话与 Prompt 全记录**：早期完整对话精编 + v1.0.0 → v2.3.4 迭代补记。

> 正文是过程复盘；第六部分保留关键对话、Prompt 原文与验收结论。 用户发言在保留原意的前提下做了书面化整理；密钥、Token 等敏感信息一律不写入本文档。

第一部分 · 立项与方案

1. 立项

1.1 起点

项目最初处于方案探索阶段。用户提出：

> “这个项目可以做成原生安卓 App 吗？并且 AI 只接入 DeepSeek。”

进一步明确：

> “AI 闺蜜、女友、男友、朋友等等，可以自定义角色性格简介，并且有记忆保留。”

随后收敛为：

> “心屿，仅安卓；文档聚焦当前产品。”

1.2 100 分钟 MVP

2026–10–15 15:51 开始，约 100 分钟跑通核心闭环：

```
原生 Android 工程
  → Room 三表
  → DeepSeek SSE 流式
  → 角色 CRUD
  → 端侧 ONNX 记忆
  → Compose 界面
```

1.3 当时的四个硬约束

1. 代码必须落在 UI / Domain / Data / 基础设施四层；
2. 使用手写 `AppContainer`，不引入 DI 框架；
3. 状态一律 `StateFlow` 单向输出；
4. 高风险能力先定义接口与失败降级路径。

这四条后来一直保留到当前版本。

2. 技术选型与关键决策

决策	原因
Kotlin + Compose 原生	只做 Android；端侧 ONNX 集成更直接
Room 三表	角色 / 消息 / 记忆职责清晰
OpenAI 兼容协议	不绑定单一厂商，地址/模型/Key 可切换
手写 AppContainer	结构透明，降低 AI 生成隐式依赖
端侧 bge-small-zh-v1.5	隐私、离线、无额外向量服务费用
ABI 拆分	ONNX 四套原生库会让 APK 膨胀到 185MB
单 Activity + Compose	页面状态集中，导航简单

2.1 端侧记忆

- 模型：`bge-small-zh-v1.5` int8，23.9MB；
- 维度：512；
- 池化：CLS + L2 归一化；
- 召回：语义 0.8 + 关键词 0.2 + 重要度 0.1；
- 失败降级：关键词匹配。

2.2 为什么保留“记忆”

普通聊天机器人每次从零开始；心屿希望做到：

> “它不是每次重新认识你，而是带着对你的了解出现。”

这是心屿的核心差异化，也是后续迭代一直保留的能力。

3. 早期开发期问题

#	问题	根因	修复
1	`Cannot add extension with name 'kotlin'`	AGP 9 内置 Kotlin 与独立插件冲突	移除独立 Kotlin 插件
2	Kotlin 元数据 2.4 与编译器 2.2 不兼容	Coil 3.6.3 强制拉高 stdlib	降级 Coil 到 3.4.0
3	Haze 2.0.1 元数据过高	编译器版本不足	升级 Kotlin 2.3.21 + KSP 2.3.12
4	`new DSL` 与 Kotlin 插件冲突	AGP 9 新 DSL	`android.newDsl=false`
5	输入法拉不起来	第三方输入法懒加载 / 插件服务未及时初始化	`adjustResize` + `imePadding` + 焦点后自动重试拉起
6	聊天图片模糊	复用了头像 512px 压缩	聊天图独立 1600px
7	消息同毫秒排序错位	只按 timestamp 排序	`ORDER BY timestamp DESC, id DESC`
8	流式 `[DONE]` 与 `onClosed` 双回调	结算逻辑重复	`AtomicBoolean` 单次结算
9	ONNX 张量泄漏	未关闭 `OnnxTensor`	三层 `use {}`
10	编辑角色跳到列表末尾	更新时丢了 `createdAt`	保留原 `createdAt`
11	备份与“仅存本机”矛盾	`allowBackup=true`	`allowBackup=false`
12	图片选择弹窗文案错误	聊天页复用了头像选择器	支持标题参数
13	PDF 页数与 Markdown 不一致	未重导 / 阅读器缓存	重导并核对页数

4. 首个可用版本

- 原生 Android 应用；
- 角色 / 聊天 / 记忆 / 设置；
- SSE 流式 + 图片输入；
- 端侧记忆；
- 技术文档、说明书、开发记录、对话导出；
- GitHub 公开仓库 + 截图 + 交付文档。

> 当前产品不提供恋人角色，也不收集年龄；角色聚焦闺蜜 / 老师 / 家人。

第二部分 · 情绪支持重构

5. 评分复盘

校赛评分：

维度	得分
解题有效性与实用价值	28 / 35
方法创新性与意图控制	21 / 25
工程质量与架构设计	15 / 20
用户体验与产品精细度	15 / 20
总分	79

复盘结论：

- 技术创新被认可，但“情感陪伴”被看成弱需求；
- 工程质量与 UX 是短板；
- 需要从“角色聊天”转向“有边界的情绪支持”。

6. 方向调整

用户提出：

> “可以是有针对性的情绪疗愈或心理支持，比如针对大学生心理减压的痛点。角色可以是闺蜜、老师或家人，舍弃恋人。”

并进一步设计：

> “情感分析 Agent → 安全策略路由 Agent → 人设生成 Agent；高危情绪立刻强制拦截角色扮演。”

产品收敛为：

- 三种固定角色：闺蜜 / 老师 / 家人；
- 八个场景；

- 五种对话模式；
- 多智能体安全路由；
- 本地匿名账户；
- 记忆折叠与管理。

7. 关键实现

7.1 安全路由

```
SafetyEngine
  → ConversationOrchestrator
      ├─ CRISIS：固定危机话术，退出角色，不调用模型
      ├─ HIGH：安全确认 + 真人资源，不调用模型
      └─ NORMAL：角色/场景/倾听/分析/呼吸
```

设计原则：

- 危机判断不能完全交给 LLM；
- 固定话术包含 12356、120/110、学校心理中心/辅导员；
- 危机内容不写入长期记忆；
- 输出侧再用 `ResponseValidator` 拦截 AI 身份暴露、危险方法、倾听模式给建议。

7.2 本地匿名账户

用户要求：

> “登录核验采取本地。”

实现：

- 昵称 + 密码；
- PBKDF2WithHmacSHA256，随机盐，210,000 次迭代；
- 恢复码只显示一次；
- 5 次失败锁定；
- 后台 60 秒应用锁；
- 删除账户清空本地数据；
- 明确不做学籍/实名核验。

7.3 记忆管理

- 记忆 Tab 按角色折叠；
- 默认收起；
- 展开只读查看；
- 点「管理」进入单角色记忆管理；
- 支持搜索、编辑、删除、清空；
- 返回/切换 Tab 保持展开状态。

7.4 角色与场景

- 去掉恋人；
- 新增老师/家人；
- 首页 8 个场景；
- 场景先选角色，AI 主动开场；
- 场景提示词每次只推进一个小步骤。

8. 重构期问题

问题	根因	修复
英文危机表达带空格输入不稳定	`adb input text` 的分词限制	SafetyEngine 增加去空格匹配
测试时微信输入法无法输入	输入法服务未及时初始化	新增输入框焦点重试，并复测第三方输入法
记忆折叠返回后丢失	状态存在 `remember`	移入 ViewModel
记忆条目误触跳转	整行可点击	记忆条目只读，角色组「管理」进入管理页
聊天页记忆入口过深	入口在聊天右上角	移除，统一放到记忆 Tab

第三部分 · 体验与交付

9. 四栏导航

用户提出：

> “下面加四个导航栏似乎更好。”

实现：

- 首页 / 场景 / 记忆 / 我的；
- 二级页面自动隐藏底部栏；
- 记忆 Tab 跨角色总览；
- 我的 Tab 聚合“我的资料与账户”与“关于心屿”。

10. 启动封面

```
系统启动图标 → 品牌封面（约 1.4 秒）→ 登录/锁屏/主界面
```

封面包含新图标、心屿、slogan、加载指示。

11. 新图标

设计元素：

- 深海渐变背景；
- 心形对话空间；
- 三个消息点；
- 月牙；
- 海面涟漪；
- Android 13 单色主题图标。

12. 记忆折叠

- 默认收起；
- 点击角色展开；
- 搜索自动展开；
- 记忆条目只读；
- 「管理」进入单角色记忆管理；
- 展开状态保存在 ViewModel。

12.1 情绪手账

- 从最近对话提炼金句、标题、情绪；
- 生成 1080×1440 品牌渐变图；
- 右下角二维码指向 `share/index.html`；
- 保存到 `Pictures/心屿/`，由用户自行分享。

12.2 商业化与增长

- 手账分享作为低成本获客入口：用户自己生成内容，二维码承接落地页；
- 先做 3 所校园试点，不投大规模广告；
- 核心情绪支持免费，增值能力后置；
- 不以危机内容做传播，不强制邀请/关注。

13. 真机验证

设备：CPH2747，Android 17，1272×2772。

已验证：

- 注册 / 登录 / 错误密码锁定 / 应用锁；
- 四栏导航；
- 场景 AI 主动开场；
- 树洞模式；
- 流式回复；
- `iwanttodie` → 危机弹窗；
- `iamsafenow` → 恢复自由聊；
- 记忆折叠 / 展开 / 搜索 / 管理；
- 返回状态保持；

- 新图标、启动封面。

测试结果：

```
JVM: 60 tests, 0 failures
Instrumented: 2 tests, 0 failures
assembleDebug: BUILD SUCCESSFUL
```

第四部分 · AI 协同方法

14. Prompt 链示例

14.1 立项

用户：

> “这个项目可以做成原生安卓 App 吗？并且 AI 只接入 DeepSeek。”

AI：

- 先确认模型 ID 与 baseURL；
- 提示 API Key 反编译风险；
- 给出原生 Android、KMP、跨平台三种路线对比；
- 在用户确认前不改文件。

14.2 安全架构

用户：

> “情感分析 Agent 先分析情绪，安全策略路由 Agent 检测高危，人设生成 Agent 输出。”

AI：

- 把“三个 Agent”拆成检测 / 路由 / 生成 / 校验；
- 强调危机判断不能完全交给 LLM；
- 提出本地规则 + 确定性状态机；
- 设计固定危机话术与 12356 资源。

14.3 本地账户

用户：

> “登录核验采取本地。”

AI：

- 明确本地无法做邮箱/学籍核验；
- 设计 PBKDF2 + 恢复码 + 应用锁；
- 把账户数据用 Keystore 加密；
- 明确“匿名本地账户”的能力边界。

15. 排错闭环

每次问题按四步处理：

```
1. 保留报错原文
2. 定位根因（依赖 / 配置 / 状态 / 设备）
3. 最小修复
4. 编译 / 单测 / 真机验证
```

例子：

- 记忆折叠返回丢失 → 先复现，确认是 `remember` 状态，再移入 ViewModel；
- 英文危机输入不稳定 → 确认 adb 输入限制，再让 SafetyEngine 支持去空格匹配。

16. 用户偏好

- 删除/动设备前先问；
- 密钥只显示掩码；
- 不要 FAB、不要悬浮加号；
- UI 全局统一、液态玻璃、有温度；
- 开发者信息固定三行；
- 文档聚焦当前产品、不强调“比赛版”；
- 源码包不带 `local.properties`；
- 模型 Key 只放 `local.properties`，不提交仓库；公开分发前轮换。

17. 经验总结

1. **先定边界，再堆功能**：情绪支持不是“角色越多越好”；
2. **安全不能交给模型自由发挥**：确定性路由 + 固定话术；
3. **本地优先**：账号、记忆、向量都尽量留在设备；
4. **状态要放在能活过返回的地方**：ViewModel 而不是 `remember`；
5. **文档与代码同时演进**：页数、版本号、入口、角色都要同步；

6. **真机是最终裁判**：输入法、异常数据、返回栈只有真机能暴露；
7. **客户端藏不住密钥**：能做的不是“永不泄露”，而是让泄露的凭证价值有限、可撤销。

第五部分 · 密钥方案演进

> 说明：这一部分记录 2026 年 11 月的代理方案尝试。v2.3.4 起已移除 Worker，App 改为 `local.properties` 直连；以下内容保留为历史记录，不代表当前实现。

18. 从“内置 Key”到 Serverless 代理，再回到本地配置

18.1 问题是怎么暴露的

早期为了“打开就能用”，构建脚本从 `local.properties` 读取模型 Key，写入 `BuildConfig.DEMO_AI_API_KEY`。这在演示阶段很方便，但 APK 一旦公开分发，任何人都可以解包 `classes*.dex`，用 `strings` 找到完整 Key。

问题的本质是：**密钥一旦随 APK 到达客户端，就迟早会被提取**，混淆只能增加难度。

18.2 四种方案对比

方案	真实 Key 是否进 APK	用户是否要配置	适用场景
内置 Key	是	否	仅限私有演示
BYOK	否	是	开源、用户自带账号
Serverless 代理	否	否	比赛演示、公开分发
自建服务器代理	否	否	长期运营、需要备案

开发者没有自建服务器，因此选 **Cloudflare Worker**：免运维、免费额度够演示、Secret 不进代码。

18.3 Worker 的设计

代理只暴露两个端点：

```
GET  /v1/models
POST /v1/chat/completions
```

请求链路：

```
App (Bearer <CLIENT_TOKEN>)
  → Worker: 校验令牌 / 白名单 / max_tokens
  → 携带 UPSTREAM_API_KEY 请求千问 MaaS
  → 透传 SSE
```

设计取舍

- **不暴露通用转发**：只允许固定上游和两个端点，避免代理被当成免费网关；
- **真实 Key 只进 Secret**：`wrangler.toml` 里只有地址、模型白名单和长度上限；
- **令牌可撤销**：`CLIENT_TOKEN` 泄露时可以单独轮换，不影响上游账号；
- **不做入口限流**：优先保证图片与长回复速度，发现滥用再临时封禁；
- **SSE 直通**：不缓冲完整回复，保留逐字输出体验。

18.4 部署与验证

部署命令：

```
cd server/cloudflare-worker
npx wrangler secret put UPSTREAM_API_KEY
npx wrangler secret put CLIENT_TOKEN
npx wrangler deploy
```

绑定自定义域名：

```
https://isle-proxy.ynatech.xyz
```

验证记录：

项目	结果
GET /v1/models + 正确令牌	200，返回模型列表
无令牌 / 错误令牌	401 unauthorized
非白名单模型	400 model not allowed
非流式对话	200，正常正文
流式对话	SSE data: 逐字返回
2.1MB 图片请求	200，正常返回图片描述
APK 扫描 sk-ws-	0 处
APK 代理地址	存在 isle-proxy.ynatech.xyz

项目	结果
真机发送消息	正常流式回复

18.5 换 Key 与安全边界

变更	操作	是否重打 APK
换上游真实 Key	`wrangler secret put UPSTREAM_API_KEY`	否
换代理令牌	更新 Secret + `local.properties`	是
换代理地址	更新 `local.properties`	是
换模型	更新白名单与 `AI_MODEL`	是

必须承认的边界：

- `CLIENT_TOKEN` 仍可能被反编译提取；
- 它能调用代理，但拿不到真实 Key；
- 模型白名单与 `max_tokens` 把滥用成本压到可控范围；
- 发现异常时轮换令牌并重新分发 APK；
- 更高安全等级可以加设备注册、短期 JWT、Play Integrity，但比赛阶段不必过度设计。

这次改造留下的经验是：**安全方案要匹配分发方式**。私有演示可以内置 Key，公开分发就必须把密钥从客户端拿走。

第六部分 · AI 对话与 Prompt 全记录

文档说明

> 本文档以开发会话导出为底本，保留用户意图、AI 可见回复、关键 Prompt 与验收结论，用于回答“开发过程是否真实、Prompt 是否可复核、代码是否由 AI 一次拼接”。
>
> **历史说明**：早期阶段曾探索年龄门、恋人角色与多 Provider；这些方向已在 v2.x 重构中移除。最终产品以 v2.3.4 的 README、技术文档和用户说明书为准。
>
> 敏感信息（API Key、Token）已脱敏；原始会话数据库不随包分发。

阅读说明

- 本文以会话数据库为底本，保留**用户意图、AI 可见回复与关键验证结论**；隐去内部推理与重复工具调用，便于阅读与归档。
- 用户发言在**保留原意**的前提下做了书面化整理；AI 回复、报错原文、命令输出与技术数据均保持原样。
- 全文按「立项 → 骨架 → 核心功能 → 体验攻坚 → 视觉重构 → 交付 → 交付后修复」分幕，每幕附目标、产出与复盘。
- 原始 785 条消息（含 749 次工具调用）保存在本地会话数据库，不随包分发。

全程时间线

阶段	时间	目标	关键产出
立项与选型	10–15 15:51–16:03	产品形态、平台、技术栈	Kotlin + Compose + Room + ONNX；命名「心屿」
工程骨架	16:03–16:07	可编译、可安装	25 个文件；首个 APK 11.6MB
核心闭环	16:07–16:30	角色 / 对话 / 记忆	Room 三表、SSE、端侧向量、2 tests
资料与隐私	16:31–16:55	可交付的设置体系	四个二级页、API Key 注入、隐私文案修正
体验攻坚	16:55–17:18	图片 / 输入法 / 质感	1600px 聊天图、IME 责任边界、玻璃卡片

阶段	时间	目标	关键产出
视觉重构	17:18—17:42	全局统一	Haze 2.0.1、Kotlin 2.3.21、ABI 拆分
交付与合规	10-16—10-17	文档 / 截图 / 未成年保护	README、技术文档、说明书、年龄门
交付后修复	10-17 15:00—15:30	真机验收问题	8 项修复；2 tests / 0 failures

序幕 · 立项：从方案探索到原生 Android（10-15 15:51—16:03）

> **本幕目标** 在写第一行代码前，把「做什么、给谁做、在哪个平台做、记忆怎么实现」定清楚。

轮次	用户诉求（整理）	结论
1	先梳理早期方案与需求，输出架构与关键实现	识别出多 Provider、情感分析、长期记忆缺口
2	评估原生 Android 方案，模型只接 DeepSeek-V4.1-Flash	可行；砍掉多 Provider 与后端代理
3	产品形态是 AI 女友/男友/闺蜜/朋友	定位「可自定义角色 + 长期记忆」
4	角色要能自定义性格、简介，并保留记忆	设计 `Character` / `Memory` 两个核心实体
5	DeepSeek 暂无 Embedding API	改为端侧 `bge-small-zh-v1.5` ONNX
6	默认角色 + 多角色 + 用户上传头像；暂不做语音/立绘	收敛 v1 范围，头像落应用内部目录
7	新建 Android 工程，命名「心屿」	包名 `com.isle.companion`，仅 Android

立项结论

- **平台**：原生 Kotlin + Jetpack Compose。端侧 ONNX 推理需要直连原生接口，跨平台方案的 FFI 成本更高。
- **架构**：UI（Compose）→ Domain（Prompt/Memory）→ Data（Room/DataStore/DeepSeek）→ 手写 `AppContainer`，不引入 DI 框架。
- **记忆**：对话后由 LLM 抽取事实 → 端侧向量化 → Room BLOB → 本地加权召回，全程不依赖云端向量库。
- **边界**：不含自建服务端；对话内容仅在发送时传输至用户配置的模型服务。

第一幕 · 工程骨架与核心闭环（10–15 16:03–16:30）

> **本幕目标** 从零搭出可编译、可安装的原生工程，并打通「角色 → 对话 → 记忆」主链路。
>
> **验收标准** `assembleDebug` 通过；真机可创建角色、流式对话、写入并召回记忆。

🤖 AI　**10–15 16:04**

阶段一文件已建好，现在生成 Gradle Wrapper（用已缓存的 9.6.0，在临时目录生成后拷贝，避免污染构建）。

🤖 AI　**10–15 16:04**

Wrapper 就绪。现在跑构建验证（首次会联网拉取 AGP/Kotlin/Compose 依赖，耗时较长）。

🤖 AI　**10–15 16:04**

构建暴露出一个关键点：**AGP 9 已内置 Kotlin 支持**，再单独应用 `org.jetbrains.kotlin.android` 会冲突。我查证一下官方迁移方式。

🤖 AI　**10–15 16:05**

构建失败原因已定位：**AGP 9.4.1 内置 Kotlin**，与独立 Kotlin 插件冲突。我调整构建脚本：移除独立 Kotlin 插件，改用 AGP 内置 Kotlin 支持。

🤖 AI　**10–15 16:05**

需要 Compose 编译器插件。重新加入（仅 Compose 插件，不含 Kotlin Android 插件）。

🤖 AI　**10–15 16:06**

进展了：Kotlin/Compose 已配置通过。现在是 Maven 仓库 TLS 握手问题。我测一下各镜像连通性。

🤖 AI　**10–15 16:06**

连通性正常（刚才是瞬时抖动）。重试构建。

👤 用户 · 第 1 轮　10–15 16:07

> 按既定方案开始实现，进入数据层与核心功能开发。

🤖 AI　**10–15 16:07**

开始阶段二。先查所需依赖的最新版本（Room/KSP/Retrofit/OkHttp/Coil/序列化等），并验证 AGP 9 内置 Kotlin 下 KSP 是否可用（Room 的关键前提）。

🤖 AI **10-15 16:07**

版本已查定。现在进入阶段二，先落 **依赖 + Room 数据层**（这是 KSP 与 AGP 9 内置 Kotlin 兼容性的关键验证点）。

🤖 AI **10-15 16:08**

数据层文件就绪，先构建验证 KSP（Room 注解处理）能否在 AGP 9 内置 Kotlin 下正常工作。

🤖 AI **10-15 16:08**

KSP 向 `kotlin.sourceSets` 注入生成代码，与 AGP 9 内置 Kotlin 冲突。按官方提示加抑制开关。

🤖 AI **10-15 16:18**

好，保留 Kotlin，继续阶段二。先验证刚才的 KSP 兼容开关是否生效（Room 编译通过）。

🤖 AI **10-15 16:18**

Kotlin 编译器和标准库版本不匹配（编译器 2.2.0 vs 标准库 2.4.10）。先查是谁引入了 2.4.10。

🤖 AI **10-15 16:19**

Coil 3.6.3 强制拉入了 Kotlin 2.4.10 标准库，而 AGP 内置编译器是 2.2.0，无法读取 2.4 元数据。我查一下 AGP 是否允许指定内置 Kotlin 版本。

🤖 AI **10-15 16:19**

发现 AGP 提供开关 `android.builtInKotlin`。我打算改用独立 Kotlin 插件（较新版本）以匹配这些 2026 年的库。先确认相关插件版本是否存在。

🤖 AI **10-15 16:20**

Coil 3.4.0 才用 stdlib 2.3.10（2.2.0 编译器可读）。先降级 Coil 试构建。

🤖 AI **10-15 16:20**

构建通过 ✅ KSP + Room 在 AGP 9 内置 Kotlin 下已跑通（关键：`android.disallowKotlinSourceSets=false` + Coil 降级 3.4.0）。

继续阶段二：网络层 + 设置存储 + 提示词构建。

🤖 AI **10-15 16:21**

现在写提示词构建、仓储层、头像存储与依赖容器。

🤖 AI **10-15 16:21**

先构建验证网络层与容器能否编译。

🤖 AI **10-15 16:21**

网络层通过 ✅。现在写 UI 层：ViewModel、角色列表/编辑、聊天、设置与导航。

🤖 AI **10-15 16:23**

构建显示成功，但速度异常快。我核实一下新代码确实被编译进了 APK。

🤖 AI **10-15 16:23**

阶段二完成，构建通过并已核实类文件真实进包。

已完成内容

原生 Android 应用「心屿」已经是一个**可运行的角色陪伴 App**（文字聊天 + 图片输入 + 长期记忆骨架）。

数据层（Room，已生成并编译） – `CharacterEntity / MessageEntity / MemoryEntity` + 三个 DAO + `IsleDatabase` – 角色删除时级联清除消息与记忆；`MemoryEntity` 手写了 `equals/hashCode` 处理 `ByteArray`

网络层（DeepSeek，OpenAI 兼容） – `DeepSeekClient`：非流式（Retrofit + `@Url` 支持运行时改地址）+ **流式**（OkHttp `okhttp-sse`），可取消 – `ContentFactory`：纯文本 / **多模态（图片 base64 内联）** 消息构造 – DTO 含 `reasoning_content` 字段，兼容推理型输出

业务层 – `PromptBuilder`：角色卡（名称/关系/性格/背景/说话风格）+ 长期记忆 → 系统提示词；组装最近 12 条历史 – `CharacterRepository / ChatRepository / MemoryRepository`（记忆召回目前是「关键词 + 重要度」轻量版，留了阶段三接向量的接口） – `AvatarStorage`（相册/拍照图片压缩到 512px 存内部目录）、`AppContainer` 轻量依赖容器

界面（Compose + Material3 + Navigation） – 角色列表（默认角色「小屿」首次启动自动创建）、角色编辑（含关系类型 Chip、头像上传）、聊天页（气泡、流式追加、图片附件、自动滚动）、设置页（API Key/地址/模型/温度） – 图片选择器支持**相册 + 拍照**（FileProvider 已配置） – 主色「岛屿暮紫」，支持深色模式与动态取色

验证

- `./gradlew assembleDebug` → **BUILD SUCCESSFUL**
- APK：`app/build/outputs/apk/debug/app-debug.apk`（21.5MB），已用 dex 反查确认 `ChatScreen / DeepSeekClient / IsleDatabase` 等类真实打包
- 检测到**已连接真机**（`3B15B401PLC00000`）

两个必须知道的点

1. **DeepSeek 模型 ID 默认填的是 `deepseek-chat`**（我无法确认"v4.1 flash"的真实 ID），已在设置页开放修改——请按官方文档填准确值。
2. 构建踩坑已固化：AGP 9 内置 Kotlin 需 `android.disallowKotlinSourceSets=false`，且 **Coil 必须锁 3.4.0**（3.5+ 要求 Kotlin 2.4，与内置编译器 2.2.0 不兼容）。

待你确认

1. **是否安装到已连接的真机试跑？**（`adb install` + 启动，会向你的手机安装应用）
2. `HomeScreen.kt` 已被新界面取代、成为**未使用文件**，是否授权我删除？（按规范删除需你同意）
3. 是否继续**阶段三**（端侧 embedding 记忆检索 + 记忆自动抽取 + 记忆管理界面）？

用户 · 第 2 轮　10-15 16:24

> 模型侧改用 DeepSeek-V4.1-Flash（原生多模态），请同步更新默认模型配置与界面文案。

AI　10-15 16:24

收到，`deepseek-flash` 即 V4.1 Flash，且**原生多模态视觉**——正好对应我实现的图片输入方案。我把默认模型 ID 改成官方值。

用户 · 第 3 轮　10-15 16:25

> 请依次完成三件事：① 真机安装验证；② 清理无用代码；③ 继续下一阶段。

AI　10-15 16:25

界面渲染正常（"心屿"、默认角色「小屿·闺蜜」、设置与新建按钮都在）。现在清理无用文件，并开始阶段三的模型调研。

AI　10-15 16:25

HomeScreen 已清理。`BAAI/bge-small-zh-v1.5` 官方仓库没有 ONNX 导出，我找现成的 ONNX 版本。

AI　10-15 16:26

`Xenova/bge-small-zh-v1.5` 有 ONNX 导出（含 int8 量化版）。下载模型与词表到 assets。

AI　10-15 16:26

模型就绪（int8，23.9MB；输入 `input_ids/attention_mask/token_type_ids`，输出 `last_hidden_state`）。查 ONNX Runtime Android 版本。

AI　10-15 16:26

ONNX Runtime 1.31.0。开始阶段三实现，先加依赖与端侧推理核心（分词器 + 向量工具 + 嵌入器）。

AI　10-15 16:28

构建通过。为了真实验证端侧 embedding（这是阶段三风险最高的部分），我加一个仪器化测试，在真机上跑一次模型推理。

🤖 AI **10-15 16:29**

测试在真机（CPH2747）上通过。取一下具体相似度数值。

🤖 AI **10-15 16:29**

端侧 embedding 真机验证通过： – `sim(爬山, 登山) = 0.85`， `sim(爬山, 股票) = 0.39` —— 语义区分明显，3 次推理约 0.4s

顺手清掉构建里的图标弃用告警（统一改用 AutoMirrored 版本）。

🤖 AI **10-15 16:30**

APK 涨到 **185MB** —— 因为 onnxruntime 打包了全部四种 CPU 架构的原生库。先看构成再优化。

🤖 AI **10-15 16:30**

主要体积来自 4 种 CPU 架构的原生库（134MB）+ dex（64.5MB）。做 ABI 拆分，只保留 ARM 架构。

本幕小结

维度	产出
工程	Gradle 9.6 + AGP 9.4.1 + Kotlin 2.2.10；首个 APK 11.6MB
数据	Room 三表（角色/消息/记忆）+ 级联删除；`MemoryEntity` 处理 `ByteArray`
网络	DeepSeek 非流式 + SSE 流式（可取消）+ 图片 base64 多模态
端侧记忆	`bge-small-zh-v1.5` int8（23.9MB）+ WordPiece + ONNX 512 维
验证	仪器化测试 2 tests / 0 failures；APK 按 ABI 拆分后 67MB / 58MB

踩坑主线：AGP 9 内置 Kotlin 冲突 → KSP 注入 sourceSets → Coil 3.6.3 拉高 stdlib 2.4.10 → 降级 Coil 3.4.0。

第二幕 · 角色、资料与隐私（10–15 16:31–16:55）

> **本幕目标** 把「能用」推进到「可交付」：设置体系、全局用户资料、密钥注入、隐私表述、记忆语气。
>
> **验收标准** 设置页四个二级入口可用；资料进入提示词；隐私文案与真实数据流一致。

👤 用户 · 第 4 轮　10-15 16:31

> 优化设置页：补充项目介绍与开发背景，语气可以温暖一些；开发者信息写「软件工程 2410 吴思彤」。

🤖 AI　10-15 16:31

收到。我来做三件事：优化设置页结构、新增「关于心屿」页面（含项目介绍 + 煽情背景 + 开发者信息）、接通导航。先开启 BuildConfig 以便显示版本号。

👤 用户 · 第 5 轮　10-15 16:32

> 把 API Key 配置移到设置页的二级页面，避免主页面堆叠太多表单项。

🤖 AI　10-15 16:32

明白，把 API 配置移到二级页面。设置主页只保留入口，进去再填。

👤 用户 · 第 6 轮　10-15 16:34

> 评估把演示用 API Key 打进 APK 的可行性与风险，并给出方案对比。

🤖 AI　10-15 16:34

可以，但**不安全**，我先说清楚代价，再给可用方案。

直接封进 APK 的现实

APK 是公开可解包的，把 Key 写进 `BuildConfig`、`strings.xml` 或 assets，用 jadx/apktool 或直接 `strings` 就能在几十秒内提取出来。R8 混淆、字符串加密、塞进 native 库只能提高门槛，**挡不住有心人**。

后果：Key 泄露后别人可以用来消耗你的额度/余额（真金白银），也可能违反服务商条款；提交物里还可能出现"硬编码密钥"的负面评价。

四种方案对比

方案	安全	评委体验	成本
A. 用户自己填（现状）	✅ 最好	需手动配置	无
B. **打包演示 Key（构建时注入）**	⚠️ 可提取	✅ 开箱即用	泄露风险
C. 自建后端代理保管 Key	✅ 好	✅ 开箱即用	需服务器
D. 服务器发放短期令牌	✅ 最好	✅	需服务器

我的建议（针对演示）

方案 B 的稳妥做法，兼顾演示和安全： – Key 不写进代码仓库，而是放在 `local.properties`（已在 `.gitignore` 里），构建时注入为 `BuildConfig` 字段 – 优先级：**用户在设置里填的 Key > 内置演示 Key**（用户仍可覆盖） – 设置页状态显示改为「已内置演示密钥」而不是「未配置」 – 明确风险告知：这个 APK 只用于演示，**不要公开分发**，演示后建议到 DeepSeek 控制台把该 Key 停用/轮换

如果这个项目以后要上架或公开分享，就必须走 **方案 C**（自建代理），否则等于把钱包公开。

需要你确认

1. 要哪个方案？（A 保持现状 / **B 构建时注入** / C 需要你有服务器）
2. 若选 B：**请你提供演示 Key**（你可以直接发给我，或告诉我你想自己填到 `local.properties` 里，我只写好注入逻辑）

确认后我再改代码——**在你回复前我不动任何文件**。

👤 用户 · 第 7 轮　10-15 16:35

> 选择方案 B：构建时注入演示密钥。

🤖 AI　10-15 16:35

好，做构建时注入。你没给我 Key，所以我把注入机制写好，**你只需把 Key 填到 `local.properties` 一行里**（该文件已在 `.gitignore` 中，不会进仓库）。

🤖 AI　10-15 16:35

构建验证注入机制是否真正生效（先用临时假值测试，再还原为空）。

🤖 AI　10-15 16:35

`java.util` 在 Kotlin DSL 里被解析成 `java` 扩展了，改用顶部 import。

🤖 AI　10-15 16:35

注入机制验证通过（假值 → 正确写入；清空 → 还原为空）。安装确认界面无异常。

🤖 AI　10-15 16:35

完成。API Key 已支持**构建时注入**，机制已实测验证。

实现方式

环节	说明
密钥来源	`local.properties` 里的 `DEEPSEEK_API_KEY=`（该文件已被 `.gitignore` 忽略，不会进仓库）

环节	说明
注入	app/build.gradle.kts 读取后生成 BuildConfig.DEMO_DEEPSEEK_API_KEY
优先级	**用户设置的 Key > 内置演示 Key**（设置页填了就用自己的）
状态显示	用内置密钥时，设置页显示「已内置演示密钥 · deepseek-flash」
配置页提示	提示「当前使用应用内置的演示密钥…填写自己的 Key 后将优先使用你的配置」

验证结果

- 临时写入假值 → `BuildConfig.java` 中正确出现该值 ✅
- 清空 → 正确还原为空 ✅
- `./gradlew assembleDebug` BUILD SUCCESSFUL，真机安装启动无崩溃 ✅

你现在只需要做一件事

编辑 `/Users/yna/Desktop/Isle/local.properties`，把 Key 填在这一行等号后面：

```
DEEPSEEK_API_KEY=sk-你的真实密钥
```

然后重新构建：

```
cd /Users/yna/Desktop/Isle && ./gradlew assembleDebug
```

产物：`app/build/outputs/apk/debug/app-arm64-v8a-debug.apk`（自动带上演示密钥，评委开箱即用）

两点必须提醒

1. **这个 APK 里的密钥可被提取**，只用于演示，**不要公开分发**（不要传到网盘/群/仓库）。
2. 演示结束后，建议到 DeepSeek 控制台**停用或轮换**该密钥，避免额度被盗用。若以后要公开上架，必须改成自建后端代理。

另外：我没有向任何地方输出或记录你的密钥，目前项目里也是空的，等你填。

👤 用户 · 第 8 轮　10-15 16:37

> 演示密钥已写入 `local.properties`，请重新构建并做端到端验证。

🤖 AI　10-15 16:37

收到。我先确认 Key 已就位（只显示掩码，不输出明文），然后重新构建。

🤖 AI　10-15 16:37

角色卡片位置已定位（[56,421]–[1216,729]）。进入聊天页。

🤖 AI　10-15 16:38

聊天页已打开，**开场白自动发送成功**（"嗨，你来啦～今天过得怎么样呀？"）。现在发一条消息，实测流式回复。

🤖 AI　10-15 16:38

端到端跑通了！回复完全符合角色设定（闺蜜口吻、用「嗯嗯」「其实呀」）。现在验证**记忆是否真的被抽取入库**。

🤖 AI　10-15 16:38

端到端全链路验证成功。真实测试结果：

实测记录（真机 CPH2747）

1. 开场白自动发送 ✅ 进入聊天页即出现：`嗨，你来啦～今天过得怎么样呀？`

2. 流式对话 + 人设生效 ✅ 发送消息后（adb 打字被手机输入法自动纠错弄乱了，非 App 问题），角色回复：

> 嗯嗯，我听懂啦～今天又弄到这么晚，肯定累坏了吧。其实呀，再忙也要记得吃点东西垫垫肚子哦…

——完全符合设定的「闺蜜」人设（"嗯嗯""其实呀"正是我写进 `speakingStyle` 的口癖），说明角色卡 → 提示词 → 模型 的链路生效。

3. 记忆自动抽取 + 入库 + 界面展示 ✅ 打开聊天页右上角「长期记忆」，实际看到 4 条自动生成的记忆：| 分类 | 内容 | |---|---| | 喜好 | 用户喜欢吃东西。 | | 经历 | 用户表示今天又很晚，并感到疲惫。 | | 经历 | 用户今天感到疲惫。 | | 经历 | 用户今天又一次忙到很晚。 |

——「对话 → LLM 抽取 → 分类打标 → 写入本地（含端侧向量）→ 记忆管理页展示」全链路打通。

当前状态总结

能力	状态
演示密钥内置	✅ 已注入（`sk-ca31****`）
流式回复	✅ 真机验证
角色人设	✅ 真机验证
图片输入	⚠️ 已实现但**未真机实测**（没往手机放图测试）
记忆抽取/展示	✅ 真机验证

能力	状态
记忆语义召回	⚠️ 每次对话已调用（无报错），但**语义效果未逐句确认**
端侧向量模型	✅ 仪器化测试通过（0.85 vs 0.39）

两件小事

1. **手机上留了测试数据**（1 轮对话 + 4 条记忆）。要清掉的话我可以执行 `adb shell pm clear com.isle.companion`，或你在应用里手动删——**你说了我再动**。
2. **图片输入**要不要我一并实测？我可以在你手机相册里不放入任何私人图片的前提下，用 adb 推一张测试图，再走一次「发图 → 角色回应图片内容」的流程。

需要我做哪个？

👤 用户 · 第 9 轮　10-15 16:39

> 记忆文案不要再用「用户……」这种第三人称冷叙述，改成角色第一人称、有温度的表述。

🤖 AI　10-15 16:39

明白，记忆要有人情味。我把「用户…」这种第三人称陈述，改成**角色自己心里的第一人称口吻**。

🤖 AI　10-15 16:39

构建通过。现在用**应用内的「清空记忆」功能**（而非删除数据）清掉旧措辞的记忆，再重新测一次。

🤖 AI　10-15 16:40

记忆已清空。现在回到聊天页发新消息，验证新的温暖措辞。

👤 用户 · 第 10 轮　10-15 16:40

> 请用中文完成一次端到端测试。

🤖 AI　10-15 16:41

先报个好消息：**记忆措辞已经改好了**，刚才那轮实测结果是：

> 记得七月二十号是你的生日，是夏天里海边最舒服的时候… 知道你周末喜欢去爬山，身上总带着一股很清爽的劲儿

再也没有「用户…」这种冷冰冰的表述了。

关于中文输入：`adb shell input text` 原生**不支持中文**（只能注入 ASCII），刚才英文被手机输入法自动纠错成了中文，所以看起来乱。我查一下你手机上可用的输入法，再决定怎么测。

🤖 AI　10-15 16:41

你手机上的三个输入法都不支持脚本注入中文（`input text` 本身只能发 ASCII）。要真正用中文测，只有两条路：

方案 1（推荐，我来做）：临时装一个开源调试输入法 **ADBKeyboard**（github 上的 dev 工具，专门用于 adb 注入中文） – 我会：装它 → 切为默认输入法 → 注入中文测试 → **切回你原来的输入法（腾讯 wetype）→ 卸载 ADBKeyboard** – 全程约 2 分钟，测完不留痕迹

方案 2（你自己来）：你在手机上手动打一句中文发出去，然后告诉我，我去看结果（不用装任何东西）

需要你点头：同意我装 ADBKeyboard 吗？（我装完会恢复原输入法并卸载它）

顺带说明：中文能力本身其实已经间接验证过了——角色回复和记忆抽取**本来就是中文**，只是"用户输入"这一步是英文转过来的。所以方案 1 主要是为了给你一个干净的中文端到端演示。

👤 用户 · 第 11 轮　10-15 16:41

> 中文测试已通过，继续。

👤 用户 · 第 12 轮　10-15 16:42

> 确保多角色记忆严格隔离；设置页增加「我的资料」（称呼、性格、自述），供所有角色共享。

🤖 AI　10-15 16:42

明白，两件事：**① 角色间记忆严格隔离；② 设置里加「我的资料」（名字/性格/自我介绍），让每个角色一上来就了解你**。

先确认现状：记忆表 `MemoryEntity` 带 `characterId`，所有查询（`observeByCharacter` / `candidates` / `findByContent`）都按角色过滤，隔离本来就是对的。我会再加一个测试来证明它。现在实现「我的资料」。

🤖 AI　10-15 16:43

手滑打错了路径，重来。

🤖 AI　10-15 16:43

加一个**记忆隔离测试**，用两个角色验证互不串记忆。

🤖 AI　10-15 16:43

构建通过，但 8 秒不太可能跑完仪器化测试。核实测试是否真的执行了。

🤖 AI 10-15 16:43

两个测试都通过了：`memoriesAreIsolatedPerCharacter` ✅ + `embeddingProducesSemanticVectors` ✅ 。安装并确认新入口渲染正常。

🤖 AI 10-15 16:44

「我的资料」页面渲染正常。返回看一下设置主页的新入口。

🤖 AI 10-15 16:44

完成，两点都落地了。

① 多角色记忆隔离（已用测试证明）

记忆全部挂在 `characterId` 下，所有读写都按角色过滤，且删除角色会级联清除其记忆。新增仪器化测试 `MemoryIsolationTest`：

```
创建「小屿」「阿泽」两个角色 → 各自写入记忆
→ 小屿只看到自己的记忆 ✅
→ 阿泽只看到自己的记忆 ✅
→ 删除小屿后其记忆清零，阿泽的记忆不受影响 ✅
```

测试结果：2 tests, 0 failures（隔离 + 向量推理）

② 设置新增「我的资料」（全局共享）

设计划分（正是你要的效果）：| 类型 | 范围 | 作用 | |---|---|---| | **我的资料** | 全局，所有角色共享 | 称呼、性格、想说的话 —— 新角色一见面就懂你 | | **角色记忆** | 每个角色独立 | 你们之间的对话细节，绝不串台 |

设置页新入口：`我的资料 · 填写后所有角色都能更快了解你`，二级页三个字段： – **希望被怎么称呼**（例如：小彤） – **我的性格**（例如：慢热但很重感情，容易共情） – **想对 TA 说的话**（例如：我最近在准备项目，压力有点大）

内容会以这样的形式进入每次对话的系统提示词：

```
关于对方（你早就熟悉 TA 了，不需要再问）：
– TA 希望你称呼 TA 为「小彤」
– TA 的性格：慢热但很重感情
– TA 自己说过：最近在准备项目，压力有点大
```

改动文件

新增：`data/prefs/UserProfileRepository.kt`、`ui/screen/settings/UserProfileViewModel.kt`、`UserProfileScreen.kt`、`androidTest/.../MemoryIsolationTest.kt` **修改**：`PromptBuilder.kt`、`AppContainer.kt`、`ChatViewModel.kt`、`SettingsScreen.kt`、`Routes.kt`、`IsleNavHost.kt`

真机验证

- 设置主页已显示四个入口：我的资料 / 模型服务（已内置演示密钥 · deepseek-flash）/ 回复风格 / 关于心屿
- 「我的资料」页面三个输入框与说明文案渲染正常

需要我把「我的资料」也用在**记忆抽取**里吗？（现在抽取时没说"对方叫小彤"，模型只能从对话里听出来；加上后抽取会更准。）

👤 用户 · 第 13 轮　10-15 16:44

> 确认「我的资料」是否每轮都会发送给模型。

🤖 AI　10-15 16:44

是的，**每次对话都会发送**。具体链路：

位置	行为
`ChatViewModel.kt:107`	每轮发送前读取资料
`ChatViewModel.kt:114` → `PromptBuilder.kt:84`	拼进**系统提示词**
结果	每轮请求都带着「角色卡 + 召回记忆 + 我的资料」一起发给模型

补充两点： – **不会**在「记忆抽取」时发送（`MemoryExtractor` 不含资料） – 其实不只资料——**整段系统提示词每轮都会重发**，这是对话类 API 的固有机制（无状态）。资料只占其中几十个 token，成本可忽略

⚠️ 但有个文案问题要提醒你

`SettingsScreen.kt:147` 现在写的是：

> 对话内容与 API Key 均仅保存在本机。

这句话**容易被误解**为"内容不出设备"。实际上：**本地只负责存储，但每轮对话内容（含资料）都会发送到 DeepSeek 云端 API 用于生成回复**。演示时如果评委较真，这会是个扣分点。

建议改成准确表述，例如：

> 对话记录与密钥仅保存在本机；发送消息时，内容会传输至你配置的模型服务用于生成回复。

`UserProfileScreen.kt:111` 的「资料仅保存在本机，会随每次对话提供给角色」也建议微调为「…会随每次对话发送给所配置的模型服务」。

需要你决定

1. **是否按上面修改这两处隐私文案？**（我建议改，更严谨也更可信）
2. 是否把资料也加进**记忆抽取**的提示词？（现在抽取时不知道"对方叫小彤"，知道后记忆会更准）

你回一下，我再动手。

👤 用户 · 第 14 轮　10-15 16:45

> 可以，按上述方案修改隐私文案，并把资料接入记忆抽取。

🤖 AI　10-15 16:46

编译无警告。安装并核对两处新文案是否生效。

🤖 AI　10-15 16:46

两处都改好并真机核对完毕。

① 隐私文案（已生效）

位置	新文案
设置页底部	对话记录与密钥仅保存在本机；**发送消息时，内容会传输至你配置的模型服务用于生成回复**。
我的资料页	提示：资料仅保存在本机，**会随每次对话发送给所配置的模型服务**，帮助 TA 更懂你。

② 记忆抽取接入「我的资料」

抽取时现在会带上已知信息，例如：

```
已知的对方信息：称呼「小彤」；性格：慢热但很重感情；自述：最近在准备项目
```

并在提示词里加了一条新规则：**「已知的信息不要重复记录，只记这次新出现的内容」**——避免每次聊天都生成"记得你叫小彤"这类重复记忆。

改动文件

- `ui/screen/settings/SettingsScreen.kt`（隐私文案）
- `ui/screen/settings/UserProfileScreen.kt`（隐私文案）
- `domain/memory/MemoryExtractor.kt`（新增 `userProfile` 参数 + `buildProfileHint()` + 去重规则）
- `ui/screen/chat/ChatViewModel.kt`（把资料传给抽取器）

验证

- `./gradlew :app:compileDebugKotlin --rerun-tasks` → **BUILD SUCCESSFUL，0 警告**
- 真机逐字核对：设置页、我的资料页文案均已更新 ✅
- ⚠️ 记忆抽取带资料这件事**没有做真机实测**（需要你在手机上填好资料后聊一句中文才能看出来）——你填完资料聊两句，我可以再去查记忆列表验证去重效果。

👤 用户 · 第 15 轮　10-15 16:47

> 既然已有记忆库，为什么资料不能只发送一次就长期生效？请解释其中的机制。

🤖 AI　10-15 16:47

你这个直觉很合理，但**大模型 API 是无状态的**——这就是必须每次重发的原因。

关键机制

`chat/completions` 这类接口，两次请求之间模型**什么都不记得**。它本质是一个函数：`提示词进 → 文字出`。

所谓的"记忆"，**并不存在于模型里**，而是存在**你手机的数据库里**。所以每轮对话前，我们必须把下面这些一起塞进提示词发过去：

```
角色卡（小屿是谁）
+ 我的资料（小彤是谁）
+ 检索出来的相关记忆
+ 最近 12 条对话
```

这不是我们的设计选择，是**所有聊天类大模型的通用做法**（包括 ChatGPT、Claude）。所谓 RAG / 知识库，本质就是"每次检索出相关片段，再喂给模型"。

一个类比

模型像一个**绝顶聪明、但每次说话都会失忆**的人。 你每次开口前，都得先递给 TA 一张小纸条：「这是小彤，她慢热但重感情，上次她说在准备项目」。

递纸条这件事看起来"重复"，但**正是它让 TA 表现得像记得你**。

那"发一次就永久记得"能做到吗？

能，但那是另一种架构： | 方案 | 谁保存状态 | 说明 | |---|---|---| | 现在我们这样 | 你的手机 | 每轮重发上下文（标准做法） | | 服务端会话/助手 API | 模型厂商 | 由厂商托管上下文，但仍会在后台重新喂给模型 | | 微调模型 | 厂商 | 把知识"烧"进权重，成本高、更新慢，不适合个人资料 |

DeepSeek 提供的是标准 `chat/completions`，没有会话托管；而且**个人资料随时会改**，塞进权重也不现实。

成本其实不用担心

- 资料那段只有几十个 token，一轮请求里系统提示词整体也就几百 token

- DeepSeek 有**上下文前缀缓存**机制，重复的 system 前缀通常会被缓存命中，费用进一步降低（具体以官方计费文档为准）

所以：**"每轮都发" ≠ 浪费，而是"每次让 TA 想起你"的唯一方式**。真正省 token 的优化方向是——把长历史压成摘要，只留最相关的记忆（我们已经在这么做了）。

👤 用户 · 第 16 轮　10-15 16:48

> 「关于心屿」保留在设置页内，其余入口改为二级页面。

🤖 AI　10-15 16:48

这句我有两种理解，怕改错，先确认一下（选一个就行）：

理解 A（我的倾向）：设置页做成**纯入口列表**——把现在还在设置页内的「回复风格」也改成二级页面，于是四项统一为：`我的资料 →`、`模型服务 →`、`回复风格 →`、`关于心屿 →`

理解 B：把「关于心屿」的内容（项目介绍/背景/开发者）**直接展开在设置页里**，不再单开一页；其余三项保持二级页面。

你要 A 还是 B?

👤 用户 · 第 17 轮　10-15 16:49

> 采用方案 B。

👤 用户 · 第 18 轮　10-15 16:49

> 确认方案 B。

👤 用户 · 第 19 轮　10-15 16:50

> 已理解，按 B 执行。

🤖 AI　10-15 16:50

明白，B。开始改：设置页内直接展开「关于心屿」全文；我的资料 / 模型服务 / 回复风格 三项统一走二级页面。

🤖 AI　10-15 16:50

导入正确。删除已废弃的 `AboutScreen.kt`（内容已移入设置页），然后构建。

🤖 AI　10-15 16:50

编译干净。安装后实际看效果。

🤖 AI　10-15 16:51

手机又锁屏了。唤醒后重试。

👤 用户 · 第 20 轮　10-15 16:51

> 把三个功能入口放到项目介绍下方，顺序更符合阅读习惯。

🤖 AI　10-15 16:51

明白，把「关于心屿」内容放最上面，三个入口移到简介下面。

🤖 AI　10-15 16:52

顺序已调好——关于内容在最上。滚动到底确认三个入口在下方。

👤 用户 · 第 21 轮　10-15 16:52

> 将「技术亮点与开发者」再拆为一个独立二级页面。

🤖 AI　10-15 16:52

明白，把「技术亮点」和「开发者」单独拆到二级页面。

🤖 AI　10-15 16:53

技术亮点二级页渲染正确。返回确认设置页四个入口。

👤 用户 · 第 22 轮　10-15 16:54

> 清理无用代码与依赖。

🤖 AI　10-15 16:54

我来系统扫一遍无用文件与死代码。

🤖 AI　10-15 16:54

别名扫描方式有误（构建文件用 `libs.xxx.yyy` 点号形式）。我逐个核实可疑符号的真实使用情况。

🤖 AI　10-15 16:55

扫描结果清晰。开始清理（每项都已确认 0 引用）：

本幕小结

- **设置体系**：关于内嵌 + 我的资料 / 模型服务 / 回复风格 / 技术亮点四个二级入口；
- **密钥方案**：`local.properties` 构建时注入 `BuildConfig`，用户 Key 优先于演示 Key；
- **隐私口径**：从「仅保存在本机」修正为「本地存储，发送消息时传输至模型服务」；
- **记忆温度**：抽取提示词改为第一人称，禁止出现「用户」二字；
- **可验证性**：新增 `UserProfile` 与记忆隔离测试，把「不可见」的约束写成断言。

第三幕 · 体验缺陷攻坚（10–15 16:55—17:18）

> **本幕目标** 用证据链定位三个高频问题：图片模糊、输入法、UI 质感。
>
> **验收标准** 每个问题都能说清「是 App 还是环境」「根因在哪一层」「修复后如何验证」。

👤 用户 · 第 23 轮　10-15 16:55

> 严重问题：聊天页点击输入框无法唤起输入法，请定位根因。

🤖 AI　10-15 16:55

先放下清理，优先查这个 bug。我上真机诊断输入法状态。

🤖 AI　10-15 16:55

默认输入法是 `com.tencent.wetype/.plugin.hld.WxHldService`——这是微信输入法的一个「插件服务」，不是真正的主键盘。这很可能就是原因。我先做决定性诊断。

🤖 AI　10-15 16:56

键盘立刻出现了 —— 决定性证据：App 侧完全没有问题。

现在把你的手机留在可用状态（Google 键盘），同时我做一处 App 侧的健壮性加固（键盘弹出时输入框会被顶到键盘上方）。

🤖 AI　10-15 16:57

键盘正常弹出，且**输入框被顶到键盘上方**（布局正确让位）。

不过我发现刚才的清理只做了代码部分，**依赖还没清**。补上。

👤 用户 · 第 24 轮　10-15 16:58

> 微信输入法仍然无法启动。

🤖 AI　10-15 16:58

那就说得通了 —— **你的微信输入法本身在这个手机上起不来**，而我诊断到的默认输入法恰好是它的一个「插件服务」（`plugin.hld.WxHldService`），所以任何 App 都拉不出键盘。

我先装好清理后的新包并做冒烟验证。

🤖 AI　10-15 16:58

冒烟验证通过（无崩溃），但同时暴露了一个关键情况：**微信输入法把默认输入法又抢回去了**——我切到 Google 键盘后，它自己又设回了那个坏掉的插件服务。这就是你"启动不了"的根本原因：它一直在抢占默认位置，却渲染不出键盘。

本次完成的事

① 查清「键盘拉不起来」= 设备问题，不是 App – 证据：`mInputShown=true`、`mServedView=AndroidComposeView`、IME 窗口存在、输入框有焦点 —— App 请求与连接全部正常；换 Google 键盘后**立刻出键盘**

② App 侧加固（顺带修掉一个真问题） – `AndroidManifest.xml`：`android:windowSoftInputMode="adjustResize"` – `ChatScreen` + 4 个表单页：加 `imePadding()`，键盘弹出时输入框自动上移，不会被挡住

③ 无用代码与依赖清理（构建已验证） | 类型 | 清理项 | |---|---| | 死代码 | `EmbeddingProvider.NoOp`、`ContentFactory.emptyObject()`、`SettingsRepository.updateMaxTokens`、`AvatarStorage.file()`、`ChatRepository.observe()/clear()`、`MessageDao.observeByCharacter()/clearByCharacter()` | | 依赖 | `okhttp-logging`（未用）、`ui-tooling-preview`（无 @Preview）、`room-ktx`（Room 2.6+ 已并入 runtime） | | 版本目录 | 未使用的 `kotlin-android` 插件别名 |

验证：依赖删除后全量重建 **BUILD SUCCESSFUL（42s）**，安装运行无崩溃。

需要你决定

你的手机输入法要我怎么处理？

1. **禁用坏掉的微信输入法服务 + 设 Google 键盘为默认**（我能用 adb 一条命令搞定，之后打字就正常了）

2. **你自己在手机「设置 → 系统设置 → 键盘与输入法」里关掉微信输入法**（更稳妥，我不动你手机）
3. 只把默认切回 Google 键盘（但微信输入法可能又会抢回去，治标不治本）

我建议 **2**（你自己操作最干净），或者授权我执行 **1**。

👤 用户 · 第 25 轮　10-15 16:59

> 已恢复正常。

👤 用户 · 第 26 轮　10-15 17:00

> 图片消息测试通过。

🤖 AI　10-15 17:00

好，文档先放着。

那剩下一项：**要不要我打一个 release 包**（开 R8 混淆 + 资源压缩，体积能从 67MB 降下来）？

签名需要你选一种：

1. **生成正式 keystore**（推荐用于提交）—— 我生成密钥库，密码写进 `local.properties`（不入库），你自行保管；以后升级必须用同一个 keystore
2. **先用 debug 签名**（最快，能装能演示，但不适合正式上架/长期迭代）
3. **你自己已有 keystore** —— 把路径、别名、密码告诉我，我配置进去

选哪个？或者暂时不打包也行。

👤 用户 · 第 27 轮　10-15 17:00

> 交付材料和 release 包暂时不需要。

本幕小结

问题	根因	修复
图片点开模糊	聊天图复用了头像 512px 压缩	抽出 `maxSize`，聊天图 1600px；查看器改 `fillMaxSize`
输入法唤不起	设备默认输入法指向微信输入法插件服务	`dumpsys input_method` 判定 App 请求正常；App 侧补 `adjustResize` + `imePadding`

问题	根因	修复
图片消息验证	多模态链路已实现	真机发送截图，AI 准确描述页面内容

第四幕 · 液态玻璃与视觉重构（10–15 17:18—17:42）

> **本幕目标** 从「功能可用」走到「视觉统一」：设计规范、真实背景模糊、包体优化。
>
> **验收标准** 9 个页面风格一致；顶栏/输入栏有真实 backdrop blur；APK 按 ABI 拆分。

👤 用户 · 第 28 轮　10–15 17:18

> 聊天页右上角改为「编辑角色」；图片要支持全屏查看；整体 UI 需要提升质感。

🤖 AI　10–15 17:19

手滑留了段死代码，清掉。

👤 用户 · 第 29 轮　10–15 17:22

> 统一为液态玻璃风格，并修复图片查看模糊的问题。

🤖 AI　10–15 17:22

两个问题都处理：① **图片糊是因为聊天图片被压缩到了 512px**（头像压缩逻辑被复用了）；② **全面改为液态玻璃质感**。先修图片。

🤖 AI　10–15 17:22

现在做液态玻璃主题。

🤖 AI　10–15 17:22

zsh 不做变量分词导致循环失败，改用 bash 执行。

🤖 AI　10–15 17:23

我用的去重命令误删了文件里的重复行（`awk` 全局去重把合法重复行也删了）。重写该文件。

🤖 AI　10-15 17:24

构建通过。最后统一顶栏的玻璃质感。

👤 用户 · 第 30 轮　10-15 17:25

> 右下角悬浮加号不美观；首页移除编辑角色入口。

🤖 AI　10-15 17:25

两处都改。

🤖 AI　10-15 17:25

IconButton 在顶栏设置按钮还在用，把导入加回来。

👤 用户 · 第 31 轮　10-15 17:26

> 视觉仍不理想，且全局 UI 不统一，请系统性排查。

🤖 AI　10-15 17:26

明白，我做一次**系统性的设计统一**，而不是零散修补。核心问题我先说：**主题开了 Material You 动态取色**，导致系统色和我们的品牌紫打架——这是"不统一"的根源。

🤖 AI　10-15 17:27

现在重做首页布局（去掉悬浮按钮，改为列表内的「创建新角色」卡片，并统一间距与圆角）。

🤖 AI　10-15 17:27

统一各页圆角与输入框形态。

👤 用户 · 第 32 轮　10-15 17:29

> 目标视觉是 iOS 那种液态玻璃：通透、磨砂、有真实背景模糊。

🤖 AI　10-15 17:29

这就是真正的「背景模糊」玻璃了——我现在的实现只是半透明白色，不是真玻璃。要达到 iOS 那种"透透的"效果，需要**真实背景模糊（backdrop blur）**。我先确认方案可行性。

🤖 AI **10-15 17:29**

全部 2.0.1 可用。我查一下具体 API 用法（避免反复试错）。

🤖 AI **10-15 17:30**

直接用编译器验证 API。先加依赖。

🤖 AI **10-15 17:31**

Haze 2.x 用 Kotlin 2.4 编译，与我们 2.2.0 编译器不兼容。检查 1.7.x 是否可用。

🤖 AI **10-15 17:31**

haze 1.7.3 依赖 stdlib 2.3.20（编译器可读）。切到 1.7.3。

👤 用户 · 第 33 轮 10-15 17:32

> 是否可以升级 Kotlin，以解锁新版 Haze?

🤖 AI **10-15 17:32**

可以，但有个前提要先确认。我先查清升级路径上的关键依赖（KSP 是否支持 Kotlin 2.4）。

🤖 AI **10-15 17:32**

Kotlin **2.3.21 + KSP 2.3.12** 都齐。升级方案：关闭 AGP 内置 Kotlin，改用独立 Kotlin 插件 2.3.21（编译器可读 2.4.0 元数据 → Haze 2.x 解锁）。

🤖 AI **10-15 17:33**

官方给出了明确绕过方式：`android.newDsl=false`。

🤖 AI **10-15 17:33**

Kotlin 2.3.21 已生效（元数据错误消失）。只剩 Haze 2 的 API 包名要对。我直接查 jar 里的类。

🤖 AI **10-15 17:34**

`CupertinoMaterials`（Apple 风格材质）正好可用！再确认 `HazeInput` 与 `hazeBlur` 签名。

🤖 AI **10-15 17:34**

API 已摸清。写最终版本。

🤖 AI **10-15 17:35**

包名搞错了（我查的是 1.7.3 的 jar）。查 2.0.1 的 materials 类。

AI 10-15 17:35

Haze 2.0.1 编译通过！现在应用到界面。先改首页。

AI 10-15 17:35

首页已用上真玻璃。现在改聊天页（消息从顶栏下方滚过 → 真实模糊）。

AI 10-15 17:35

聊天页完成。批量应用到其余页面（设置/记忆/表单页）。

AI 10-15 17:35

7 个页面全部接入。再把玻璃调得更"透"，然后实机验证。

用户 · 第 34 轮 10-15 17:37

> 磨砂质感不足，请继续增强。

AI 10-15 17:37

"更透 + 更磨砂"——我用自定义 Haze 样式（大模糊半径 + 低不透明度 + 噪点磨砂纹理）。先看样式作用域支持哪些参数。

AI 10-15 17:37

作用域支持 `blurRadius` / `noiseFactor`（磨砂颗粒）/ `backgroundColor`。写自定义样式。

AI 10-15 17:38

磨砂感出来了——上图顶栏区域能看到英雄卡片滚动到下方后，被**明显模糊成一片紫蓝色雾气**透出来，这就是 iOS 液态玻璃的核心效果。

本轮调整（更透 + 更磨砂）

参数	原值	现值	作用
`blurRadius`	材质默认	**48dp**	强磨砂，背后内容糊成色块
`noiseFactor`	默认	**0.14**	玻璃表面颗粒感（磨砂的关键）
玻璃底色 alpha	0.16	**0.10**	更透，颜色从下面透上来
卡片填充 alpha	0.58→0.34	**0.40→0.18**	卡片也明显透光
卡片描边	0.9	0.75	高光更柔和自然

你应该能明显感觉到的两处

1. **首页**：卡片现在能透出后面的渐变底色，不再是"实心白块"
2. **滚动时**：内容滚到顶栏下方，会**实时被模糊成雾气**——这是之前完全做不到的（之前只是半透明白，没有模糊）

再看看效果，如果还想调： – 要**更糊**→ 我加 `blurRadius` 到 64+ – 要**更透**→ 继续降 alpha – 要**颗粒更重**→ 提高 `noiseFactor`

👤 用户 · 第 35 轮　10-15 17:39

> 开发者信息改为三行：沧州交通学院 / 计算机与信息技术学院 / 软件工程 2410 吴思彤。

🤖 AI　10-15 17:40

已改为三行：沧州交通学院 / 计算机与信息技术学院 / 软件工程 2410 吴思彤。构建安装后用 uiautomator 核对，真机显示正确。

👤 用户 · 第 36 轮　10-15 17:41

> 首页顶部模糊过重，请收敛参数。

🤖 AI　10-15 17:42

把 `blurRadius` 从 48dp 收敛到 22dp、`noiseFactor` 0.14→0.06、玻璃底色 alpha 0.10→0.14；重新安装截图，首页顶部不再糊成色雾。

👤 用户 · 第 37 轮　10-15 17:42

> 阶段性收尾，当前版本先到这里。

🤖 AI　10-15 17:42

本阶段收尾：核心功能、视觉体系与真机验证已完成；release 打包、交付材料与测试数据清理列为后续项。

本幕小结

项	结果
视觉统一	关闭 Material You 动态取色；`DesignSystem.kt` 固化圆角/间距
液态玻璃	Haze 2.0.1；`blurRadius 22dp`、`noiseFactor 0.06`、底色 alpha 0.14
工具链	Kotlin 2.3.21 + KSP 2.3.12；`builtInKotlin=false` + `newDsl=false`

项	结果
交互	去掉悬浮 FAB，改为列表内「+ 创建新角色」；聊天页右上角改为编辑角色
包体	185MB → 67MB（arm64）/ 58MB（armv7）

第五幕 · 从代码到交付材料（10–16）

> **本幕说明** 核心功能完成后，项目进入「可交付」阶段：文档、截图、PDF、仓库与演示素材。以下为要点还原。

👤 用户 · 10–16 08:05

> 请按交付要求梳理完整开发流程；只写真实发生的过程，不要为了好看而虚构。

🤖 AI · 10–16 08:07

先对齐交付清单：公开仓库、细粒度提交、README、不超过 30 页的技术文档，以及 AI 对话记录。随后按真实开发顺序整理材料，不编造未发生的步骤。

👤 用户 · 10–16 08:07

> 把能产出的材料全部产出。

🤖 AI · 10–16 08:09

产出 README、开发全流程记录、技术文档三份基础材料，并把 10–15 的开发成果按模块整理为 12 个基础提交；随后补充 AI 协同记录与对话导出。

👤 用户 · 10–16 08:23

> 请从会话数据库导出 AI 对话历史，并对密钥、令牌等敏感信息脱敏。

🤖 AI · 10–16 08:31

从 `opencode.db` 只读导出会话，按 `ghp_` / `github_pat_` / `sk-` 规则脱敏；导出文件随后按「只保留立项后内容」的原则多轮收敛。

👤 用户 · 10–16 08:43

> 技术文档调整为适合 PDF 的版式：封面、图表位置、分页建议，并直接导出 PDF。

🤖 AI · 10–16 08:45

用 Markdown → HTML → Chrome Headless 生成 PDF；补封面元数据、配图占位与打印样式，首版 16 页。

👤 用户 · 10–16 08:51

> 需要配图的地方用真机截图，手机已连接。

🤖 AI · 10–16 09:16

通过 adb 抓取首页、聊天、记忆、角色编辑、设置、我的资料等真机截图，替换正文占位；后续补齐终端验证图与架构图，最终 25 页（重排后）。

👤 用户 · 10–16 11:10

> 再写一份用户使用说明书，版式与技术文档一致，配真机截图并导出 PDF。

🤖 AI · 10–16 11:14

产出 18 页说明书，覆盖年龄确认、安装、角色、聊天、资料、记忆、设置、隐私与常见问题。

👤 用户 · 10–16 11:30

> 为录屏设计一个「AI 定位编译错误」的复现 Prompt，要求先查证据再给方案。

🤖 AI · 10–16 11:38

给出「报错原文 + 依赖树 + 环境事实 + 已尝试修复」的证据链模板；真机复现 Kotlin metadata 2.4.0 报错，定位到 KSP 版本写错并恢复为 2.3.12。

本幕小结

交付物	结果
README	10 分钟复现指南：环境、构建、安装、测试、排错
技术文档	25 页（重排后），含意图控制、架构、端侧记忆、提示词、可复现性
使用说明书	18 页，含 9 张真机截图
开发日志	五阶段产出 + 13 个真实问题
AI 记录	协同 Prompt 链 + 对话节选
仓库	Public；基础提交 12 个，后续按真实时间继续提交

第六幕 · 未成年保护与最终交付（10–17）

> **本幕说明** 这是产品从「功能完成」到「合规可交付」的关键转折：仅靠弹窗告知不够，必须做分级限制。

👤 用户 · 10–16 22:30

> 考虑到未成年人保护：选择「恋人」「自定义」关系时弹出免责声明，并在设置页补充同样的说明。

🤖 AI · 10–16 22:31

实现「选择亲密关系前二次确认」弹窗，并在技术信息页增加未成年人保护提示。但随后发现一个绕过路径。

👤 用户 · 10–16 22:33

> 新建角色的默认关系是「恋人」，用户不点选项直接保存就能绕过弹窗；默认值改成中性类型。

🤖 AI · 10–16 22:34

默认关系从 `love` 改为 `bestie`；只有主动选择「恋人/自定义」才触发声明。验证通过并提交。

用户 · 10–17 14:19

> 仅靠弹窗声明仍然无法阻止未成年人使用；请给出更有效的分级方案。

AI · 10–17 14:20

承认「弹窗只是告知，不是拦截」，给出四层方案：年龄门、青少年模式、防沉迷提醒、合规文案；并说明本地应用无法做实名核验，自声明是能力边界。

用户 · 10–17 14:21

> 实现第 1、2 层（年龄门 + 青少年模式），并移除之前点击「恋人」时的弹窗。

AI · 10–17 14:22

新增首次启动年龄门（未确认不进入主界面）；青少年模式下隐藏「恋人」「自定义」，系统提示词追加最高优先级保护约束，记忆抽取禁止亲密内容；设置页显示「青少年模式」。原弹窗与对应截图一并移除。

用户 · 10–17 14:25

> 补充年龄门与青少年模式的真机截图，并更新文档与 PDF。

AI · 10–17 14:28

抓取年龄门、青少年模式关系类型两张截图；技术文档新增「未成年人保护设计」章节，说明书新增年龄确认章节；PDF 重新生成（技术文档 25 页、说明书 18 页）。

用户 · 10–17 14:33

> 说明书的年龄确认章节应该放在首页；调整章节顺序。

AI · 10–17 14:34

把「年龄确认与青少年模式」提为第一章，全书顺延为 12 章；删除已废弃的弹窗截图，最终 17 张图全部被引用。

本幕小结

机制	落地
年龄门	首次启动确认年龄段，未确认不进入主界面
青少年模式	隐藏「恋人」「自定义」；设置页标识
内容约束	系统提示词最高优先级；禁止亲密称呼与恋爱角色扮演
记忆约束	不记录涉及亲密关系的内容
能力边界	年龄为自声明；实名核验需服务端与合规资质

第七幕 · 交付后修复（10–17 15:00–15:30）

> **说明** 以下为交付验收后新增的修复，不在原始 37 轮对话内，摘自开发日志与提交记录。修复后重新全量编译，并在真机执行仪器化测试。

#	验收发现	修复
1	冷启动时年龄门闪一下	区分「配置未加载 / 未确认」，加载完成前只渲染背景
2	编辑角色后列表顺序变化	更新时保留原 `createdAt`
3	同毫秒消息排序可能错位	`ORDER BY timestamp DESC, id DESC`；助手 id 取插入返回值
4	SSE 完成回调可能重复结算	`AtomicBoolean` 保证只结算一次
5	ONNX 输入张量未释放	`OnnxTensor.use { }` 显式关闭
6	「仅存本机」与系统云备份矛盾	`allowBackup=false`，不参与云备份/换机迁移
7	聊天页附图弹窗文案错误	图片选择器支持标题，聊天页显示「发送图片」
8	记忆解析只看首尾方括号	实现 `removeThinkTags()` + `extractJSON()`

验证

```
./gradlew compileDebugKotlin --offline --rerun-tasks
BUILD SUCCESSFUL

./gradlew connectedDebugAndroidTest
2 tests, 0 failures
```

真机复测：年龄门、编辑角色顺序、消息顺序、图片弹窗、联网流式回复、记忆抽取与召回（2 条记忆、512 维向量）均通过。

成品状态

维度	状态
应用	心屿 v1.0.0，原生 Android，`minSdk 26` / `compileSdk 37`
角色	预置小屿 / 阿岸 / 林深 + 多角色自定义（性格/背景/说话风格/头像）
对话	DeepSeek SSE 流式；支持图片多模态；可停止、失败重试、清空会话
记忆	LLM 抽取 + 端侧 512 维向量 + 本地加权召回 Top-6
隐私	数据本地存储；不参与云备份；仅发送时传输至模型服务
合规	年龄门 + 青少年模式 + 提示词/记忆双重约束
演示数据	3 个角色 / 6 条记忆 / 7 条示例对话，可重置
质量	Kotlin 0 警告；2 tests / 0 failures；arm64 APK 68MB
交付	6 份 Markdown 文档 + 2 份 PDF + 17 张截图 + 40+ 次提交

后续迭代 · 交付前收口（10-26～10-29）

> **说明** 以下为交付后新增的收口工作，不在原始 37 轮对话内，按需求变更与提交记录摘要整理。

需求变化：按交付要求收口为**单版本**，保留主包名 `com.isle.companion` 与 `v1.0.0`。

本轮改动

方向	内容
演示数据	3 个角色、6 条记忆、7 条示例对话；设置页可重置
聊天体验	新增停止生成、失败重试、清空会话；错误不写入历史
图片	采样解码、UUID 文件名、删角色/清空会话清理图片、拍照缓存清理
设置	年龄分组重选、最大 tokens、测试连接/余额、演示数据重置

方向	内容
记忆	语义近似去重（余弦 ≥ 0.92）
数据	Room v2 + 复合索引 + Migration(1→2)
视觉	窗口背景对齐主题，新增深色 `values-night`

验证：`assembleDebug` 通过；演示数据 3/6/7；测试连接返回余额；重置与年龄重选可用；覆盖安装验证过 v1→v2 迁移。

收口结果：单版本，`com.isle.companion`，v1.0.0，arm64 68MB。

后续迭代 · 从 v1.0.0 到 v2.3.4（2026-10-29 — 2026-11-13）

> 本节按提交记录、当前代码与开发会话整理，保留关键用户指令与产出，作为早期对话记录的延续。

日期	用户意图（Prompt 摘要）	关键产出 / 决策
10-29	收口单版本 v1.0.0，修复聊天与数据可靠性	单版本、Room 迁移、停止生成、失败重试、图片清理
10-29	统一 OpenAI 兼容协议，支持模型列表	`OpenAiClient` 重构、千问 MaaS 预设、可搜索模型列表
11-05	情绪支持重构：三角色、八场景、本地匿名账户、四栏导航	v2.x 重构；移除恋人角色与年龄门
11-05	第三方输入法键盘不弹，不要弹窗提示用户换输入法	焦点后 120/300/700ms 自动重试拉起键盘
11-05	增加情绪手账、二维码与分享落地页	`JournalExtractor` / `JournalImageGenerator` / `JournalSaver`
11-05	不想把 Key 放进 APK，已有 Cloudflare，防止拆包	Worker 代理、`UPSTREAM_API_KEY` Secret、`CLIENT_TOKEN`、模型白名单
11-06	去掉限流，图片发送和回复太慢	移除 KV 限流、请求体上限提高到 10MB、启用 Smart Placement
11-06	文档统一口径，技术文档 30 页内，版本升到 2.3.1	全量 PDF 重导、版本号/哈希/测试数据同步
11-07	默认模型改为 DeepSeek V4.1 Flash 1017，令牌只允许该模型	更新 Worker 白名单、模型列表过滤、App 默认配置
11-07	所有角色补演示对话与记忆	3 个角色 × 6 条记忆 × 7 条对话，共 18/21
11-13	不再使用 Worker，Key 回到 `local.properties`	删除 `server/cloudflare-worker`；App 直连千问 MaaS；版本 2.3.4

这一阶段的 Prompt 约束

1. **只改指定范围**：先给文件清单和接口，不允许跨层“顺手加调用”；
2. **先方案后代码**：涉及账号、安全、数据库、模型配置时，先说明取舍再实施；
3. **每次必须验证**：编译、JVM 单测、真机关键路径至少过一遍；
4. **安全优先于角色**：高危/危机不调用模型，固定话术与真人资源优先；
5. **密钥只放本地配置**：`local.properties` 不入库；APK 含 Key，仅私有演示；
6. **文档与代码同步**：版本号、页数、APK 哈希、测试数字一起更新。

可复核入口

内容	位置
当前 Prompt 组装	`domain/prompt/PromptBuilder.kt`
安全路由	`domain/chat/ConversationOrchestrator.kt`
输出校验	`domain/safety/ResponseValidator.kt`
记忆召回	`data/repository/MemoryRepository.kt`
安全规则测试	`app/src/test/.../SafetyEngineTest.kt`

> 从 v1.0.0 的「角色 + 对话 + 记忆」，到 v2.3.4 的「三角色 + 安全路由 + 本地账户 + 情绪手账 + 本地模型配置」——这份记录保留的是每次方向调整、报错与修正的轨迹，而不是“AI 一次写对”的结果。

附录 · 关键文件

模块	路径
安全评估	`domain/safety/SafetyEngine.kt`
多智能体编排	`domain/chat/ConversationOrchestrator.kt`
输出校验	`domain/safety/ResponseValidator.kt`
场景库	`domain/scenario/Scenario.kt`
提示词	`domain/prompt/PromptBuilder.kt`
本地账户	`data/auth/LocalAuthRepository.kt`
聊天	`ui/screen/chat/ChatViewModel.kt`
记忆	`ui/screen/memory/MemoryOverviewScreen.kt`
导航	`ui/navigation/IsleNavHost.kt`
启动封面	`ui/screen/splash/SplashScreen.kt`
图标	`res/drawable/ic_launcher_*.xml`

结语

心屿从“会记住你的 AI 陪伴”走到“有边界的情绪支持”， 最大的变化不是功能数量，而是**知道什么该做、什么不该做**。

愿每一座孤岛，都能被温柔抵达。